Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій
Кафедра оптоелектроніки та інформаційних технологій

Звіт
До лабораторної роботи №4
Варіант 9
На тему
“Рекурсія та мемоізація в Python”

Виконав:
Студент групи ФЕП-22с
Обиход І. В.
Перевірив:
Ас. Остріков Д. А.

Львів 2026

Мета роботи:

- зрозуміти рекурсивні обчислення та структуру рекурсивних алгоритмів;
- розібратися з обмеженнями стеку в Python і відсутністю TCO;
- навчитися мемоізувати обчислення за допомогою functools.lru_cache;
- реалізувати quicksort, обходи дерев, memoized Fibonacci.

Теоретичні відомості:

- Рекурсивна функція складається з бази рекурсії (простий випадок) і рекурсивного кроку (розбиття на менші підзадачі).
- У CPython ліміт глибини стеку близько 1000 кадрів, після чого виникає RecursionError.
- Оптимізації хвостових викликів (TCO) у CPython немає, тому для глибоких структур безпечніші ітеративні алгоритми.
- Мемоізація (@lru_cache) кешує результати для однакових аргументів і прибирає повторні обчислення підзадач.

Завдання (варіант 9 - Лабіринт):

Рекурсивним DFS знайти шлях у прямокутному лабіринті від старту до фінішу. Не мутувати вхідну карту; порівняти з ітеративним DFS/BFS.

Хід роботи

Базову частину реалізовано в algorithms.py: чистий quicksort з випадковим pivot, вузол дерева Node і рекурсивні обходи preorder, inorder, postorder на генераторах, ітеративні bfs_level_order і dfs_preorder_iter, висота дерева рекурсивно та ітеративно, fib з @lru_cache, fib_naive, fib_iter і хвостовий fact.

Варіант 9 реалізовано в maze.py. Лабіринт - незмінний tuple рядків. dfs_path_recursive має базовий випадок «поточна клітинка - фініш» і рекурсивний крок «спробувати кожного невідвіданого сусіда»; множина відвіданих клітинок (frozenset) передається в рекурсію і повертається з неї, а не мутується.

Як ітеративні альтернативи реалізовано dfs_path_iterative з власним стеком і bfs_shortest_path з чергою deque, яка знаходить найкоротший шлях. render_path повертає нову карту з позначеним шляхом.

Для демонстрації мемоізації реалізовано підрахунок кількості шляхів кроками вниз і вправо: count_paths_naive без кешу та count_paths_memo через @lru_cache з cache_info() і cache_clear().

Тести перевіряють коректність шляхів, відсутність шляху, незмінність карти, збіг мемоізованої і наївної версій та RecursionError рекурсивного DFS на коридорі довжиною 3000 клітинок, який ітеративна версія проходить без проблем.

Файл maze.py

maze.py рядки 1-56

```python
"""Варіант 9 ЛР4: пошук шляху в лабіринті рекурсією та ітерацією.

Лабіринт - tuple рядків: '#' стіна, '.' прохід, 'S' старт, 'F' фініш.
Вхідна карта ніколи не змінюється.
"""

from collections import deque
from collections.abc import Iterable
from functools import lru_cache
from itertools import pairwise

type Cell = tuple[int, int]
type Maze = tuple[str, ...]
type Path = tuple[Cell, ...]

WALL = "#"
START = "S"
FINISH = "F"
MOVES: tuple[Cell, ...] = ((0, 1), (1, 0), (0, -1), (-1, 0))


def parse_maze(lines: Iterable[str]) -> Maze:
    """Перетворює рядки на незмінний прямокутний лабіринт."""
    maze = tuple(line.strip() for line in lines if line.strip())
    if not maze or len({len(row) for row in maze}) != 1:
        raise ValueError("Лабіринт має бути непорожнім і прямокутним")
    return maze


def find(maze: Maze, ch: str) -> Cell:
    """Знаходить координати символу."""
    for r, row in enumerate(maze):
        c = row.find(ch)
        if c >= 0:
            return r, c
    raise ValueError(f"Символ {ch!r} не знайдено")


def is_open(maze: Maze, cell: Cell) -> bool:
    """Чи клітинка в межах карти і не є стіною."""
    r, c = cell
    return 0 <= r < len(maze) and 0 <= c < len(maze[0]) and maze[r][c] != WALL


def neighbors(maze: Maze, cell: Cell) -> tuple[Cell, ...]:
    """Сусідні прохідні клітинки."""
    r, c = cell
    return tuple(
        (r + dr, c + dc) for dr, dc in MOVES if is_open(maze, (r + dr, c + dc))
    )


def dfs_path_recursive(maze: Maze) -> Path | None:
    """Рекурсивний DFS. Множина відвіданих передається далі, а не мутується."""
    start, finish = find(maze, START), find(maze, FINISH)
```

maze.py рядки 57-111

```python
    def walk(cell: Cell, seen: frozenset[Cell]) -> tuple[Path | None, frozenset[Cell]]:
        """База: фініш. Крок: спробувати кожного невідвіданого сусіда."""
        if cell == finish:
            return (cell,), seen
        for nxt in neighbors(maze, cell):
            if nxt in seen:
                continue
            rest, seen = walk(nxt, seen | {nxt})
            if rest is not None:
                return (cell, *rest), seen
        return None, seen

    return walk(start, frozenset({start}))[0]


def dfs_path_iterative(maze: Maze) -> Path | None:
    """Ітеративний DFS з власним стеком (не впирається в ліміт рекурсії)."""
    start, finish = find(maze, START), find(maze, FINISH)
    stack: list[Path] = [(start,)]
    seen = {start}
    while stack:
        path = stack.pop()
        if path[-1] == finish:
            return path
        for nxt in reversed(neighbors(maze, path[-1])):
            if nxt not in seen:
                seen.add(nxt)
                stack.append((*path, nxt))
    return None


def bfs_shortest_path(maze: Maze) -> Path | None:
    """BFS: найкоротший шлях за кількістю кроків."""
    start, finish = find(maze, START), find(maze, FINISH)
    queue: deque[Path] = deque([(start,)])
    seen = {start}
    while queue:
        path = queue.popleft()
        if path[-1] == finish:
            return path
        for nxt in neighbors(maze, path[-1]):
            if nxt not in seen:
                seen.add(nxt)
                queue.append((*path, nxt))
    return None


def is_valid_path(maze: Maze, path: Path) -> bool:
    """Перевіряє, що шлях іде від S до F сусідніми прохідними клітинками."""
    if not path or path[0] != find(maze, START) or path[-1] != find(maze, FINISH):
        return False
    steps_ok = all(abs(a[0] - b[0]) + abs(a[1] - b[1]) == 1 for a, b in pairwise(path))
    return steps_ok and all(is_open(maze, cell) for cell in path)
```

maze.py рядки 112-161

```python
def render_path(maze: Maze, path: Path) -> Maze:
    """Повертає НОВУ карту, де шлях позначено '*'."""
    marks = set(path)
    return tuple(
        "".join(
            "*" if (r, c) in marks and ch not in (START, FINISH) else ch
            for c, ch in enumerate(row)
        )
        for r, row in enumerate(maze)
    )


def count_paths_naive(maze: Maze) -> int:
    """Кількість шляхів S->F кроками вниз/вправо без кешу (експоненційно)."""
    finish = find(maze, FINISH)

    def go(r: int, c: int) -> int:
        """База: стіна/вихід за межі - 0, фініш - 1."""
        if not is_open(maze, (r, c)):
            return 0
        if (r, c) == finish:
            return 1
        return go(r + 1, c) + go(r, c + 1)

    return go(*find(maze, START))


@lru_cache(maxsize=None)
def count_paths_from(maze: Maze, r: int, c: int) -> int:
    """Мемоізована кількість шляхів з (r, c) до фінішу кроками вниз/вправо."""
    if not is_open(maze, (r, c)):
        return 0
    if (r, c) == find(maze, FINISH):
        return 1
    return count_paths_from(maze, r + 1, c) + count_paths_from(maze, r, c + 1)


def count_paths_memo(maze: Maze) -> int:
    """Кількість шляхів S->F з мемоізацією через lru_cache."""
    return count_paths_from(maze, *find(maze, START))


def open_grid(rows: int, cols: int) -> Maze:
    """Порожній лабіринт rows x cols зі стартом у лівому верхньому кутку."""
    middle = "." * cols
    first = START + middle[1:]
    last = middle[:-1] + FINISH
    if rows == 1:
        return (START + "." * (cols - 2) + FINISH,)
    return (first, *([middle] * (rows - 2)), last)
```

Файл algorithms.py

algorithms.py рядки 1-56

```python
"""Базова частина ЛР4: quicksort, обходи дерева, мемоізований Фібоначчі."""

from collections import deque
from collections.abc import Iterator, Sequence
from dataclasses import dataclass
from functools import lru_cache
from random import choice


def quicksort(xs: Sequence[int]) -> list[int]:
    """Чистий рекурсивний quicksort з випадковим pivot (вхід не змінюється)."""
    if len(xs) <= 1:
        return list(xs)
    pivot = choice(xs)
    left = [x for x in xs if x < pivot]
    mid = [x for x in xs if x == pivot]
    right = [x for x in xs if x > pivot]
    return quicksort(left) + mid + quicksort(right)


@dataclass(frozen=True, slots=True)
class Node:
    """Вузол бінарного дерева."""

    key: int
    left: "Node | None" = None
    right: "Node | None" = None


def preorder(t: Node | None) -> Iterator[int]:
    """Рекурсивний прямий обхід: корінь, ліве, праве."""
    if t is None:
        return
    yield t.key
    yield from preorder(t.left)
    yield from preorder(t.right)


def inorder(t: Node | None) -> Iterator[int]:
    """Рекурсивний симетричний обхід: ліве, корінь, праве."""
    if t is None:
        return
    yield from inorder(t.left)
    yield t.key
    yield from inorder(t.right)


def postorder(t: Node | None) -> Iterator[int]:
    """Рекурсивний зворотний обхід: ліве, праве, корінь."""
    if t is None:
        return
    yield from postorder(t.left)
    yield from postorder(t.right)
    yield t.key
```

algorithms.py рядки 57-114

```python
def bfs_level_order(t: Node | None) -> list[int]:
    """Ітеративний обхід у ширину через чергу."""
    if t is None:
        return []
    queue, out = deque([t]), []
    while queue:
        node = queue.popleft()
        out.append(node.key)
        queue.extend(child for child in (node.left, node.right) if child)
    return out


def dfs_preorder_iter(t: Node | None) -> list[int]:
    """Ітеративний прямий обхід через власний стек."""
    if t is None:
        return []
    stack, out = [t], []
    while stack:
        node = stack.pop()
        out.append(node.key)
        stack.extend(child for child in (node.right, node.left) if child)
    return out


def height(t: Node | None) -> int:
    """Висота дерева рекурсивно."""
    if t is None:
        return 0
    return 1 + max(height(t.left), height(t.right))


def height_iter(t: Node | None) -> int:
    """Висота дерева ітеративно (обхід рівнями)."""
    level = [t] if t else []
    depth = 0
    while level:
        depth += 1
        level = [c for n in level for c in (n.left, n.right) if c]
    return depth


@lru_cache(maxsize=None)
def fib(n: int) -> int:
    """Мемоізований рекурсивний Фібоначчі: O(n) замість O(2^n)."""
    if n < 0:
        raise ValueError("n має бути невід'ємним")
    if n < 2:
        return n
    return fib(n - 1) + fib(n - 2)


def fib_naive(n: int) -> int:
    """Рекурсивний Фібоначчі без кешу (експоненційний)."""
    if n < 2:
        return n
    return fib_naive(n - 1) + fib_naive(n - 2)
```

algorithms.py рядки 115-129

```python
def fib_iter(n: int) -> int:
    """Ітеративний Фібоначчі без рекурсії та стеку."""
    if n < 0:
        raise ValueError("n має бути невід'ємним")
    a, b = 0, 1
    for _ in range(n):
        a, b = b, a + b
    return a


def fact(n: int, acc: int = 1) -> int:
    """Хвостова рекурсія. CPython не має TCO, тож стек усе одно росте."""
    if n <= 1:
        return acc
    return fact(n - 1, acc * n)
```

Файл main.py

main.py рядки 1-51

```python
"""Точка входу лабораторної роботи №4, варіант 9."""

import sys
import time
from collections.abc import Callable

from algorithms import fib, fib_iter, fib_naive, quicksort
from maze import (
    bfs_shortest_path,
    count_paths_from,
    count_paths_memo,
    count_paths_naive,
    dfs_path_iterative,
    dfs_path_recursive,
    open_grid,
    parse_maze,
    render_path,
)

MAZE_TEXT = """
S.#.......
.##.####..
....#..#.#
#.#.#.##..
..#...#..F
"""


def measure(fn: Callable[[], object], repeat: int = 1) -> float:
    """Повертає час виконання в мілісекундах."""
    start = time.perf_counter()
    for _ in range(repeat):
        fn()
    return (time.perf_counter() - start) * 1000 / repeat


def run() -> None:
    """Демонструє пошук шляху та порівняння рекурсії, ітерації і мемоізації."""
    maze = parse_maze(MAZE_TEXT.splitlines())
    print("Лабіринт:")
    print("\n".join(maze))
    for name, finder in [
        ("рекурсивний DFS", dfs_path_recursive),
        ("ітеративний DFS", dfs_path_iterative),
        ("BFS (найкоротший)", bfs_shortest_path),
    ]:
        path = finder(maze)
        assert path is not None
        print(f"\n{name}: {len(path) - 1} кроків")
        print("\n".join(render_path(maze, path)))
```

main.py рядки 52-76

```
    print(f"\nЛіміт рекурсії CPython: {sys.getrecursionlimit()}")
    grid = open_grid(11, 11)
    count_paths_from.cache_clear()
    t_naive = measure(lambda: count_paths_naive(grid))
    t_memo = measure(lambda: count_paths_memo(grid))
    print(f"Шляхів 11x11 вниз/вправо: {count_paths_memo(grid)}")
    print(f"  без кешу: {t_naive:.2f} мс, з lru_cache: {t_memo:.2f} мс")
    print(f"  {count_paths_from.cache_info()}")
    t_rec = measure(lambda: dfs_path_recursive(open_grid(30, 30)), 20)
    t_it = measure(lambda: dfs_path_iterative(open_grid(30, 30)), 20)
    print(f"DFS 30x30: рекурсивний {t_rec:.2f} мс, ітеративний {t_it:.2f} мс")
    print(f"fib(25): naive {measure(lambda: fib_naive(25)):.2f} мс, ", end="")
    fib.cache_clear()
    print(f"lru_cache {measure(lambda: fib(25)):.3f} мс, ", end="")
    print(f"iter {measure(lambda: fib_iter(25)):.3f} мс")
    print(f"quicksort([5, 3, 8, 1, 9, 2]) = {quicksort([5, 3, 8, 1, 9, 2])}")


def main() -> None:
    """Запуск демонстрації рекурсії та мемоізації."""
    run()


if __name__ == "__main__":
    main()
```

Файл tests/test_maze.py

test_maze.py рядки 1-58

```python
"""Тести варіанта 9: лабіринт."""

from collections.abc import Callable

import pytest

from maze import (
    Maze,
    Path,
    bfs_shortest_path,
    count_paths_from,
    count_paths_memo,
    count_paths_naive,
    dfs_path_iterative,
    dfs_path_recursive,
    is_valid_path,
    open_grid,
    parse_maze,
    render_path,
)

MAZE: Maze = parse_maze(
    [
        "S.#....",
        ".##.##.",
        "...#...",
        "#.....F",
    ]
)

Finder = Callable[[Maze], Path | None]
FINDERS: list[Finder] = [dfs_path_recursive, dfs_path_iterative, bfs_shortest_path]


@pytest.mark.parametrize("finder", FINDERS)
def test_path_found_and_valid(finder: Finder) -> None:
    """Тест: кожен алгоритм знаходить коректний шлях."""
    path = finder(MAZE)
    assert path is not None
    assert is_valid_path(MAZE, path)


def test_bfs_is_shortest() -> None:
    """Тест: BFS дає шлях не довший за DFS."""
    bfs = bfs_shortest_path(MAZE)
    dfs = dfs_path_recursive(MAZE)
    assert bfs is not None and dfs is not None
    assert len(bfs) == 10
    assert len(bfs) <= len(dfs)


@pytest.mark.parametrize("finder", FINDERS)
def test_no_path(finder: Finder) -> None:
    """Тест: якщо фініш відрізано стіною, шляху немає."""
    blocked = parse_maze(["S.#F", "..#."])
    assert finder(blocked) is None
```

test_maze.py рядки 59-106

```python
def test_start_is_finish_neighbor() -> None:
    """Тест: найпростіший випадок - фініш поруч зі стартом."""
    tiny = parse_maze(["SF"])
    assert dfs_path_recursive(tiny) == ((0, 0), (0, 1))


def test_input_not_mutated() -> None:
    """Тест: пошук і рендер не змінюють вхідну карту."""
    before = tuple(MAZE)
    path = dfs_path_recursive(MAZE)
    assert path is not None
    marked = render_path(MAZE, path)
    assert MAZE == before
    assert marked != MAZE and "*" in "".join(marked)


def test_parse_maze_errors() -> None:
    """Тест: непрямокутний і порожній лабіринт - помилка."""
    with pytest.raises(ValueError):
        parse_maze(["S..", ".F"])
    with pytest.raises(ValueError):
        parse_maze([])


def test_count_paths_memo_matches_naive() -> None:
    """Тест: мемоізована версія дає той самий результат, що й наївна."""
    grid = open_grid(6, 6)
    assert count_paths_naive(grid) == count_paths_memo(grid) == 252
    assert count_paths_memo(MAZE) == count_paths_naive(MAZE)


def test_cache_info_and_clear() -> None:
    """Тест: cache_info показує влучання, cache_clear очищає кеш."""
    count_paths_from.cache_clear()
    count_paths_memo(open_grid(5, 5))
    info = count_paths_from.cache_info()
    assert info.hits > 0 and info.currsize > 0
    count_paths_from.cache_clear()
    assert count_paths_from.cache_info().currsize == 0


def test_deep_maze_recursion_limit() -> None:
    """Тест: глибокий коридор ламає рекурсію, ітеративна версія працює."""
    corridor = open_grid(1, 3000)
    with pytest.raises(RecursionError):
        dfs_path_recursive(corridor)
    path = dfs_path_iterative(corridor)
    assert path is not None and len(path) == 3000
```

Файл tests/test_quicksort.py

test_quicksort.py рядки 1-26

```python
"""Тести quicksort."""

import random

from algorithms import quicksort


def test_quicksort_basic() -> None:
    """Тест: базові та граничні випадки."""
    assert quicksort([]) == []
    assert quicksort([1]) == [1]
    assert quicksort([3, 2, 1]) == [1, 2, 3]
    assert quicksort([2, 2, 1, 3, 2]) == [1, 2, 2, 2, 3]


def test_quicksort_random() -> None:
    """Тест: збіг із sorted на випадкових даних."""
    xs = [random.randint(-1000, 1000) for _ in range(1000)]
    assert quicksort(xs) == sorted(xs)


def test_quicksort_does_not_mutate() -> None:
    """Тест: вхідний список не змінюється."""
    xs = [5, 1, 4]
    quicksort(xs)
    assert xs == [5, 1, 4]
```

Файл tests/test_tree.py

test_tree.py рядки 1-40

```python
"""Тести обходів дерева."""

from algorithms import (
    Node,
    bfs_level_order,
    dfs_preorder_iter,
    height,
    height_iter,
    inorder,
    postorder,
    preorder,
)


def sample_tree() -> Node:
    """Дерево: 4 -> (2 -> 1, 3), (6 -> -, 7)."""
    return Node(4, left=Node(2, Node(1), Node(3)), right=Node(6, None, Node(7)))


def test_traversals() -> None:
    """Тест: послідовності рекурсивних та ітеративних обходів."""
    t = sample_tree()
    assert list(preorder(t)) == [4, 2, 1, 3, 6, 7]
    assert list(inorder(t)) == [1, 2, 3, 4, 6, 7]
    assert list(postorder(t)) == [1, 3, 2, 7, 6, 4]
    assert bfs_level_order(t) == [4, 2, 6, 1, 3, 7]
    assert dfs_preorder_iter(t) == [4, 2, 1, 3, 6, 7]


def test_empty_tree() -> None:
    """Тест: порожнє дерево."""
    assert list(preorder(None)) == []
    assert bfs_level_order(None) == []
    assert dfs_preorder_iter(None) == []
    assert height(None) == height_iter(None) == 0


def test_height() -> None:
    """Тест: висота рекурсивно та ітеративно."""
    assert height(sample_tree()) == height_iter(sample_tree()) == 3
```

Файл tests/test_fib.py

test_fib.py рядки 1-44

```python
"""Тести Фібоначчі та хвостової рекурсії."""

import pytest

from algorithms import fact, fib, fib_iter, fib_naive


def test_fib_small() -> None:
    """Тест: перші 10 значень."""
    assert [fib(i) for i in range(10)] == [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]


def test_fib_iter_matches() -> None:
    """Тест: мемоізована, наївна та ітеративна версії збігаються."""
    for n in range(50):
        assert fib(n) == fib_iter(n)
    assert fib_naive(15) == fib(15)


def test_cache_clear_and_info() -> None:
    """Тест: API кешу lru_cache."""
    fib.cache_clear()
    fib(30)
    assert fib.cache_info().currsize == 31
    fib(30)
    assert fib.cache_info().hits >= 1
    fib.cache_clear()
    assert fib.cache_info().currsize == 0


def test_negative_input() -> None:
    """Тест: від'ємний аргумент - ValueError."""
    with pytest.raises(ValueError):
        fib(-1)
    with pytest.raises(ValueError):
        fib_iter(-1)


def test_fact_tail_recursion() -> None:
    """Тест: факторіал і відсутність TCO на великій глибині."""
    assert fact(5) == 120
    assert fact(0) == 1
    with pytest.raises(RecursionError):
        fact(100_000)
```

Запуск програми



Рис. 1. Вивід програми (python main.py)

Виконання тестів

```
Windows PowerShell

PS D:\IPZ2kurs\func\lab1\Lab_04> py -m pytest -v
============================= test session starts =============================
platform win32 -- Python 3.13.9, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\LENOVO\AppData\Local\Programs\Python\Python313\python.exe
rootdir: D:\IPZ2kurs\func\lab1\Lab_04
configfile: pyproject.toml
testpaths: tests
plugins: platformdirs-4.12.4
collecting ... collected 24 items

tests/test_fib.py::test_fib_small PASSED                                 [  4%]
tests/test_fib.py::test_fib_iter_matches PASSED                          [  8%]
tests/test_fib.py::test_cache_clear_and_info PASSED                      [ 12%]
tests/test_fib.py::test_negative_input PASSED                            [ 16%]
tests/test_fib.py::test_fact_tail_recursion PASSED                       [ 20%]
tests/test_maze.py::test_path_found_and_valid[dfs_path_recursive] PASSED [ 25%]
tests/test_maze.py::test_path_found_and_valid[dfs_path_iterative] PASSED [ 29%]
tests/test_maze.py::test_path_found_and_valid[bfs_shortest_path] PASSED  [ 33%]
tests/test_maze.py::test_bfs_is_shortest PASSED                          [ 37%]
tests/test_maze.py::test_no_path[dfs_path_recursive] PASSED              [ 41%]
tests/test_maze.py::test_no_path[dfs_path_iterative] PASSED              [ 45%]
tests/test_maze.py::test_no_path[bfs_shortest_path] PASSED               [ 50%]
tests/test_maze.py::test_start_is_finish_neighbor PASSED                 [ 54%]
tests/test_maze.py::test_input_not_mutated PASSED                        [ 58%]
tests/test_maze.py::test_parse_maze_errors PASSED                        [ 62%]
tests/test_maze.py::test_count_paths_memo_matches_naive PASSED           [ 66%]
tests/test_maze.py::test_cache_info_and_clear PASSED                     [ 70%]
tests/test_maze.py::test_deep_maze_recursion_limit PASSED                [ 75%]
tests/test_quicksort.py::test_quicksort_basic PASSED                     [ 79%]
tests/test_quicksort.py::test_quicksort_random PASSED                    [ 83%]
tests/test_quicksort.py::test_quicksort_does_not_mutate PASSED           [ 87%]
tests/test_tree.py::test_traversals PASSED                               [ 91%]
tests/test_tree.py::test_empty_tree PASSED                               [ 95%]
tests/test_tree.py::test_height PASSED                                   [100%]

============================= 24 passed in 0.14s ==============================
```

Рис. 2. Результат запуску pytest

Перевірка типів і стилю

```
Windows PowerShell

PS D:\IPZ2kurs\func\lab1\Lab_04> py -m mypy .
Success: no issues found in 7 source files
PS D:\IPZ2kurs\func\lab1\Lab_04> py -m ruff check .
All checks passed!
PS D:\IPZ2kurs\func\lab1\Lab_04> py -m black --check .
All done!
7 files would be left unchanged.
```

Рис. 3. mypy, ruff та black без зауважень

Висновок:

Під час виконання роботи реалізовано рекурсивний і ітеративні алгоритми пошуку шляху в лабіринті та базові рекурсивні алгоритми. Рекурсивні функції мають чіткі базові випадки і не змінюють вхідних даних. Мемоізація через lru_cache скоротила підрахунок шляхів у сітці 11x11 із сотень мілісекунд до долей мілісекунди. Тести показали, що через ліміт стеку і відсутність TCO глибока рекурсія в CPython падає, тож для великих структур слід обирати ітеративні версії.